

COAD: Constant-Time Planning for Continuous Goal Manipulation with Compressed Library and Online Adaptation

Anonymous

Abstract—In many robotic manipulation tasks, the robot repeatedly solves motion-planning problems that differ mainly in the location of the goal object and its associated obstacle, while the surrounding workspace remains fixed. Prior works have shown that leveraging experience and offline computation can accelerate repeated planning queries, but they lack guarantees of covering the continuous task space and require storing large libraries of solutions. In this work, we present COAD, a framework that provides constant-time planning over a continuous goal-parameterized task space. COAD discretizes the continuous task space into finitely many Task Coverage Regions. Instead of planning and storing solutions for every region offline, it constructs a compressed library by only solving representative root problems. Other problems are handled through fast adaptation from these root solutions. At query time, the system retrieves a root motion in constant time and adapts it to the desired goal using lightweight adaptation modules such as linear interpolation, Dynamic Movement Primitives, or simple trajectory optimization. We evaluate the framework on various manipulators and environments in simulation and the real world, showing that COAD achieves substantial compression of the motion library while maintaining high success rates and sub-millisecond-level queries, outperforming baseline methods in both efficiency and path quality. The source code is available at <https://github.com/elpis-lab/CoAd>.

I. INTRODUCTION

Motion planning is a central problem in robotics, enabling autonomous systems to plan collision-free motions while satisfying kinematic and geometric constraints [1]. In robotic manipulation, this problem becomes particularly challenging due to high-dimensional configuration spaces, complex robot geometry, and cluttered workspaces. Despite these challenges, significant progress has been made over the years [2], and recent efforts have focused on leveraging experience and pre-computation to accelerate repeated queries [3].

Library-based and experience-based [3]–[6] planners store and adapt previously computed solutions, achieving substantial speedups. However, despite their practical effectiveness, these methods typically provide no guarantees that all task instances drawn from a continuous distribution will be solvable.

In this work, we consider a structured yet practically important setting in which the distribution of planning problems is known. Specifically, we study manipulation tasks such as the tabletop picking scenario shown in Fig. 1, where a robot must repeatedly grasp a known object placed at varying poses while other workspace obstacles remain fixed. Such settings commonly arise in production workcells for tasks such as kitting, packaging, and pick-and-place. We refer to this family of problems as **goal-varying motion planning**, where each planning query differs only in the pose of a known object within a bounded region of $SE(3)$.

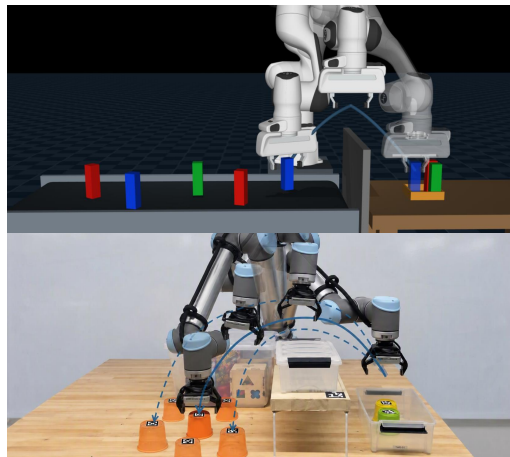


Fig. 1: Robots repeatedly solve motion-planning problems to grasp an object whose pose continuously varies across queries, while the rest of the workspace remains unchanged.

This setting induces an infinite family of motion planning problems due to the *continuous goal* pose. COAD addresses this by leveraging the following key property: if a path is valid for a nominal object pose and remains valid under bounded perturbations of that pose, then that single path certifies feasibility for an entire continuous subset of nearby object placements. This property allows the continuous goal space to be covered by finitely many regions, where a single path certifies feasibility for all goals within each region. Furthermore, rather than computing a distinct motion plan for every region, COAD constructs a *compressed library* of root paths. The remaining regions are handled through fast *adaptation* of these root paths toward their region goals, while preserving feasibility within their certified coverage. At query time, COAD identifies the region containing the query goal, retrieves the corresponding root path, and adapts it online.

To the best of our knowledge, this is the first framework that provides constant-time guarantees for continuous goal-parameterized motion planning domains. Prior constant-time approaches assume a discrete representation of the goal space [7]. Concretely, our contributions are:

- A partitioning framework that enumerates an infinite family of planning queries to a finite set of paths, each associated with a certified region of goal poses.
- We demonstrate how an adapt and reuse framework can compress the number of paths while preserving coverage of the continuous task domain.
- We demonstrate the effectiveness of COAD on manipulation tasks with 7-DOF and 8-DOF manipulators, showing compression of up to 95% of the full path library while achieving query times as low as 0.05 ms.

II. RELATED WORK

A. Classical Motion Planning

Motion planning has been studied extensively for several decades. Existing planners broadly fall into three categories. Sampling-based motion planners (SBMPs), such as PRM [8] and RRT-Connect [9], are *probabilistically complete*: if a solution exists, they will find it with probability one as the computation time approaches infinity. However, they provide no deterministic runtime guarantees. Graph-based planners are *resolution complete*, guaranteeing a solution at a fixed discretization resolution, but they scale poorly in high-dimensional manipulation spaces. Optimization-based planners [10], [11] often achieve fast performance in practice but lack global guarantees and may converge to local minima. While these methods are general-purpose and applicable to arbitrary motion planning problems, they do not provide guarantees over structured families of problems drawn from a continuous task distribution.

B. Experience-Based Motion Planning

Experience-based motion planning seeks to accelerate repeated queries by equipping robots with memory and reusing prior solutions in related environments. Rather than planning from scratch, these methods retrieve and adapt stored trajectories [3], [4], [12], roadmaps or learned sampling strategies [13].

These approaches have demonstrated substantial speedups in challenging manipulation tasks, and our method adopts a similar retrieve-and-adapt strategy to CoverLib [3]. However, while these methods operate over continuous problem domains, they do not provide deterministic guarantees that the entire continuous task domain is covered.

C. Constant-Time and Precomputation-Based Planning

More closely related to our work are fixed-time motion planning approaches [7], [14]–[16], which shift computational effort to an offline phase to guarantee bounded query-time performance. These methods typically pre-compute solutions over a finite or pre-discretized set of task or obstacle configurations. In particular [7] iteratively select uncovered discrete goals, compute root paths, and reuse them to cover nearby goals. Our library construction adopts this general coverage-based strategy. The key distinction is that we first derive a finite, certifiably covered representation from an underlying continuous goal space. Our framework therefore provides constant-time guarantees over a continuous goal-parameterized domain. The work in [15] also operates over a continuous domain, but considers varying obstacles with a fixed goal, whereas we consider a continuously varying goal and its associated obstacle.

III. NOTATIONS AND PROBLEM STATEMENT

Let $q \in \mathcal{C}$ denote the configuration and configuration space of a robot. Let $\mathcal{C}_{\text{obs}}$ denote the obstacle (invalid) configuration space, and denote the collision-free space as $\mathcal{C}_{\text{free}} = \mathcal{C} \setminus \mathcal{C}_{\text{obs}}$.

A. Motion Planning Problem

Given a start configuration $q_{\text{start}} \in \mathcal{C}_{\text{free}}$ and a goal region $\mathcal{C}_{\text{goal}} \subseteq \mathcal{C}_{\text{free}}$, motion planning problem tries to find a continuous path $\pi : [0, 1] \rightarrow \mathcal{C}_{\text{free}}$ such that $\pi(0) = q_{\text{start}}$, $\pi(1) \in \mathcal{C}_{\text{goal}}$.

B. Goal-varying motion planning problem

In this work, we study a class of manipulation planning problems in which the goal depends on the pose of a movable object. Let $\mathbf{T}_o \in \mathcal{T} \subseteq SE(3)$ denote the pose of a target object o , whose placement varies across queries within the bounded task space $\mathcal{T}$ while the environment remains static. Such settings are common in tasks such as robotic grasping, where the robot must reach a grasp pose relative to the object (Fig. 1). For each object pose $\mathbf{T}_o$, the task induces a goal region in the robot configuration space, denoted $\mathcal{C}_{\text{goal}}(\mathbf{T}_o)$. Because the object itself occupies space in the workspace, its pose also affects the set of valid robot configurations. Consequently, the collision-free configuration space becomes object-dependent and is denoted $\mathcal{C}_{\text{free}}(\mathbf{T}_o)$.

Given an object pose $\mathbf{T}_o \in \mathcal{T}$, a start configuration $q_{\text{start}} \in \mathcal{C}_{\text{free}}(\mathbf{T}_o)$ and a goal region $\mathcal{C}_{\text{goal}}(\mathbf{T}_o)$, we aim to compute a continuous path $\pi : [0, 1] \rightarrow \mathcal{C}_{\text{free}}(\mathbf{T}_o)$ such that $\pi(0) = q_{\text{start}}$, $\pi(1) \in \mathcal{C}_{\text{goal}}(\mathbf{T}_o)$, if such a path exists. We denote the family of these problems as:

$$p(\mathcal{T}) = \{(q_{\text{start}}, \mathcal{C}_{\text{goal}}(\mathbf{T}_o), \mathcal{C}_{\text{free}}(\mathbf{T}_o)) | \mathbf{T}_o \in \mathcal{T}\}$$

where both the goal region and the collision-free space vary continuously with $\mathbf{T}_o$.

IV. METHODOLOGY

Since the set of motion planning problems $p(\mathcal{T})$ contains infinitely many instances, computing a unique path for each is infeasible. Instead, we leverage the observation that a single path can remain valid for a continuous subset of object poses.

In Sec. IV-A, we show how this property allows us to partition the task space into a finite set of regions. In Sec. IV-B and Sec. IV-C, we describe how COAD leverages these finite regions to construct a motion-plan library. Finally, we show in Sec. IV-D that it enables constant-time solution for any problem $\mathbf{T}_o \in \mathcal{T}$. The overall framework is illustrated in Fig. 2.

A. Task Space Discretization

1) *Task Space Region*: We begin by describing the goal regions $\mathcal{C}_{\text{goal}}(\mathbf{T}_o)$ induced by the pose of the object using the *Task Space Regions* (TSRs) [17]. A TSR defines a set of admissible end-effector poses relative to an object that are valid for completing the task (e.g., grasping), as in Fig. 3 (a).

Formally, let $\mathbf{T}_\delta \in SE(3)$ denote a displacement transform bounded by a box $\mathbf{B} \subset \mathbb{R}^6$ specifying translational and rotational limits. The resulting displacement set is denoted $\Delta(\mathbf{B})$. Let $\mathbf{T}_s \in SE(3)$ denote a fixed end-effector offset relative to the object frame, and let $\mathbf{T}_o$ denote the pose of object o . The corresponding TSR is defined as:

$$\text{TSR}(\mathbf{T}_o) = \{\mathbf{T}_e | \exists \mathbf{T}_\delta \in \Delta(\mathbf{B}) \text{ s.t. } \mathbf{T}_e = \mathbf{T}_o \mathbf{T}_\delta \mathbf{T}_s\}. \quad (1)$$

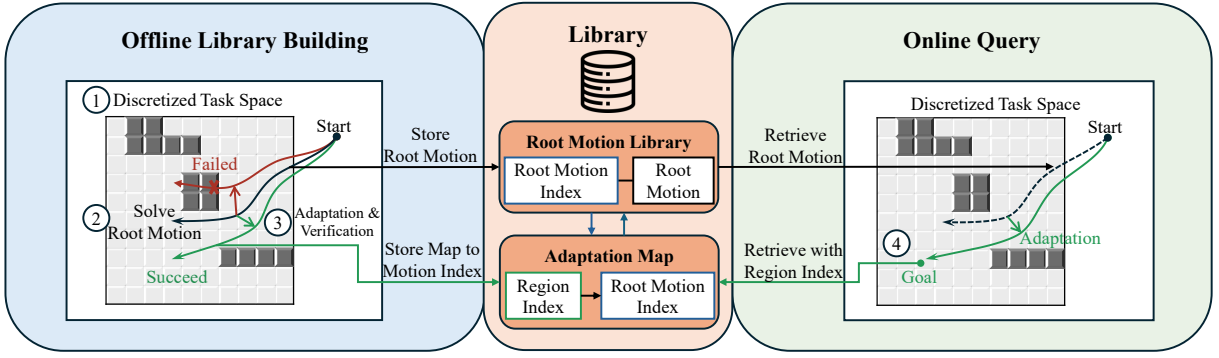


Fig. 2: COAD consists of an offline library-building stage and an online query stage. 1) The continuous task space is discretized into a finite set of TCRs, each indexed for constant-time lookup. 2) During offline preprocessing, an uncovered region is selected, a motion planner finds a *root motion* for that region, and the resulting path is stored in the root-motion library. 3) The same root motion is then adapted and verified on neighboring regions; successful adaptations are recorded in an adaptation map that links each covered region index to a root-motion index. 4) During the online query stage, the sensed Goal pose is mapped to its region index, the corresponding root motion is retrieved in constant time, and the motion is adapted to satisfy the queried goal condition using an adaption algorithm. This process yields a compressed library that provides constant plans and covers the full task space.

Geometrically, the TSR is generated by allowing the local offset pose $\mathbf{T}_\delta$ to vary over a 6-dimensional box, and mapping each such offset to an end-effector pose via rigid body transformations $\mathbf{T}_e = \mathbf{T}_o \mathbf{T}_\delta \mathbf{T}_s$. Thus, in the local coordinates of $\mathbf{T}_\delta$, the TSR is exactly a box-shaped region.

The goal region in configuration space is therefore:

$$\mathcal{C}_{\text{goal}}(\mathbf{T}_o) = \{q \mid \text{FK}(q) \in \text{TSR}(\mathbf{T}_o)\}, \quad (2)$$

where FK denotes the forward kinematics of the robot.

2) *Shared goals across object poses*: Consider two sufficiently close object poses $\mathbf{T}_{o1}$ and $\mathbf{T}_{o2}$ whose TSRs overlap, as illustrated in Fig. 3 (b). In this case, there exists an end-effector pose $\mathbf{T}_e$ that lies in the intersection of the two TSRs and therefore constitutes a valid goal for both object placements.

More importantly, in the example shown in Fig. 3 (c), the shared end-effector pose remains valid throughout the illustrated transformation between $\mathbf{T}_{o1}$ and $\mathbf{T}_{o2}$. This observation implies that a single end-effector goal can be valid for continuous subset of object poses.

3) *Task Coverage Regions*: Motivated by this observation, we introduce the notion of a Task Coverage Region (TCR). Rather than associating each object pose with a set of valid end-effector poses (as in TSRs), we invert this relationship and define the set of object poses that a fixed end-effector pose can solve.

Definition 1. (Task Coverage Region):

Given a TSR with $\mathbf{T}_\delta$, $\Delta(\mathbf{B})$ and $\mathbf{T}_s$, we define the Task Coverage Region (TCR) of an end-effector pose $\mathbf{T}_e$ as the set of object poses for which $\mathbf{T}_e$ lies within their TSRs:

$$\text{TCR}(\mathbf{T}_e) = \{\mathbf{T}_o \mid \mathbf{T}_e \in \text{TSR}(\mathbf{T}_o)\}. \quad (3)$$

Substituting the TSR definition from Eq. 1 yields

$$\text{TCR}(\mathbf{T}_e) = \{\mathbf{T}_e \mathbf{T}_s^{-1} \mathbf{T}_\delta^{-1} \mid \mathbf{T}_\delta \in \Delta(\mathbf{B})\}. \quad (4)$$

Intuitively, a TCR describes the region of object poses in $\mathcal{T}$ for which a fixed end-effector pose $\mathbf{T}_e$ remains valid. Geometrically, a TCR is obtained by allowing the local offset pose $\mathbf{T}_\delta$ to vary over the same 6-dimensional box and solving

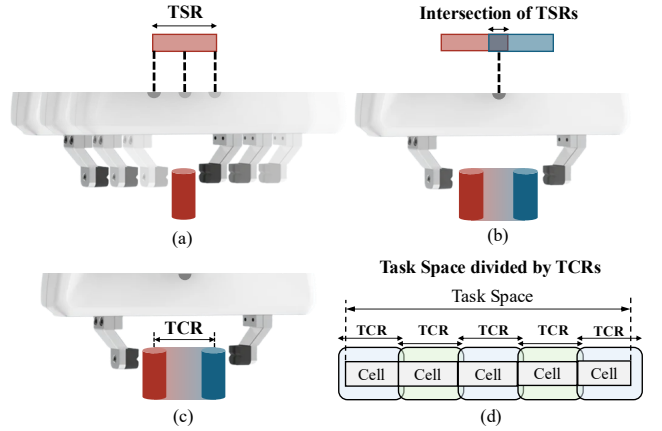


Fig. 3: 1D illustration of TSRs and TCRs.

for the corresponding object pose through: $\mathbf{T}_o = \mathbf{T}_e \mathbf{T}_s^{-1} \mathbf{T}_\delta^{-1}$. However, due to the inversion this region is not an axis-aligned box in the object-pose local coordinates of $\mathcal{T}$.

a) *Finite coverage of the task space*: Our goal is to represent the continuous task space $\mathcal{T}$ using a finite set of representative end-effector goals. Specifically, we seek a set of end effector poses $\{\mathbf{T}_e^i\}_{i=1}^N$ such that every task pose $\mathbf{T}_o \in \mathcal{T}$ lies within at least one coverage region:

$$\mathcal{T} \subseteq \bigcup_{i=1}^N \text{TCR}(\mathbf{T}_e^i). \quad (5)$$

Each representative end-effector pose $\mathbf{T}_e^i$ is therefore valid for every object pose contained in its corresponding TCR.

Since TCRs may have irregular shapes and may overlap, we discretize the task space $\mathcal{T}$ into axis-aligned cells to enable constant-time indexing. Each cell must lie entirely within one TCR so that a single representative end-effector pose $\mathbf{T}_e$ is valid for every task pose in that cell, as illustrated in Fig. 3 (d).

We first establish that such non-degenerate local cells exist. Specifically, whenever the TSR displacement set $\Delta(\mathbf{B})$ has non-empty interior, every TCR also has non-empty interior and therefore contains an axis-aligned box under a local task-space coordinate chart.

Theorem 1. (Local TCR coverage):

Let $\Delta(\mathbf{B}) \subset SE(3)$ be the bounded displacement set of a TSR. If $\Delta(\mathbf{B})$ has non-empty interior, then for any fixed $\mathbf{T}_e, \mathbf{T}_s \in SE(3)$, $\text{TCR}(\mathbf{T}_e)$ also has non-empty interior. Consequently, under any valid local coordinate chart, $\text{TCR}(\mathbf{T}_e)$ contains a non-degenerate axis-aligned box $\mathbf{B}_{\text{in}}$.

Proof. From Eq. 4, define the map $f(\mathbf{T}) = \mathbf{T}_e \mathbf{T}_s^{-1} \mathbf{T}^{-1}$. Matrix inversion and left multiplication by a fixed rigid transform are diffeomorphisms on $SE(3)$, hence f is also a diffeomorphism and therefore maps open sets to open sets. Since $\Delta(\mathbf{B})$ has non-empty interior, it contains a non-empty open set U . Then $f(U)$ is open and satisfies $f(U) \subseteq \text{TCR}(\mathbf{T}_e)$. Therefore, $\text{TCR}(\mathbf{T}_e)$ contains a non-empty open set and has non-empty interior. Under any valid local task-space coordinate chart, this open set corresponds to a non-empty open subset of $\mathbb{R}^6$, which contains an axis-aligned box with non-empty interior. $\square$

Theorem 1 establishes local TCR coverage. If the identity transform $\mathbf{I}$ lies in the interior of $\Delta(\mathbf{B})$, then for any task pose $\mathbf{T}_o$, choosing $\mathbf{T}_e = \mathbf{T}_o \mathbf{T}_s$ gives $\mathbf{T}_o = \mathbf{T}_e \mathbf{T}_s^{-1} \mathbf{I}^{-1}$, so $\mathbf{T}_o$ lies in the interior of $\text{TCR}(\mathbf{T}_e)$. Hence the TCRs form an open cover of the task space, and for a compact task domain, a finite subcover exists.

Although such inner regions always exist, deriving an explicit axis-aligned box for general $SE(3)$ variations is difficult because inversion couples translation and rotation. We therefore first consider the most common manipulation setting in which objects vary in (x, y, z, ψ) , where ψ is yaw, and roll and pitch remain fixed.

Let the TSR bounds be $|\Delta x| \leq b_x, |\Delta y| \leq b_y, |\Delta z| \leq b_z$, and $|\Delta \psi| \leq b_\psi$. After inversion, planar translation is coupled with yaw. A conservative yaw-independent inner bound is obtained by

$$h_{xy} = \frac{\min(b_x, b_y)}{\sqrt{2}}, \quad (6)$$

since any point satisfying $|\Delta x|, |\Delta y| \leq h_{xy}$ has planar norm no greater than $\min(b_x, b_y)$ and therefore remains inside the admissible translation region under any yaw rotation. The z and yaw components remain decoupled, yielding the inner-box widths:

$$\delta_x = 2h_{xy}, \quad \delta_y = 2h_{xy}, \quad \delta_z = 2b_z, \quad \delta_\psi = 2b_\psi. \quad (7)$$

These widths define a regular grid over the bounded task space. For $x \in [x_l, x_h], y \in [y_l, y_h], z \in [z_l, z_h]$, and wrapped yaw $\bar{\psi} \in [0, 2\pi)$, a query pose is indexed by

$$i_x = \lfloor \frac{x - x_l}{\delta_x} \rfloor, i_y = \lfloor \frac{y - y_l}{\delta_y} \rfloor, i_z = \lfloor \frac{z - z_l}{\delta_z} \rfloor, i_\psi = \lfloor \frac{\bar{\psi}}{\delta_\psi} \rfloor. \quad (8)$$

For each cell r , let $\mathbf{T}_o^r$ be its center and define $\mathbf{T}_e^r = \mathbf{T}_o^r \mathbf{T}_s$. By construction, the entire cell lies within $\text{TCR}(\mathbf{T}_e^r)$, and the index tuple uniquely indexes a TCR cell and can be used as a key to retrieve end-effector goals and motion solutions.

For more general task variations, including full $SE(3)$ poses or additional articulated-object parameters, we use a heuristic yet practical numerical procedure to determine

candidate inner-box dimensions. Starting from a nominal pose $\mathbf{T}_o$ with $\mathbf{T}_e = \mathbf{T}_o \mathbf{T}_s$, we perturb the object pose along each task-space dimension and sample poses within the resulting candidate region to evaluate whether $\mathbf{T}_e$ remains inside the corresponding TSR.

For every constructed cell, we additionally check the associated robot goal configuration against a conservative enclosure of the goal-object geometry over the entire cell. If this check fails, the cell bounds are reduced until the collision constraint is satisfied.

B. Offline Library Building with Compression

Algorithm 1: BuildLibrary($\mathcal{R}, \mathcal{A}, q_{\text{start}}$)

```

/* Initialize library */
1  $L \leftarrow$  Empty root motion list;
2  $\mathcal{M} \leftarrow$  Empty hash map from a TCR  $r$  to root motion
    $\pi_{\text{root}}$  and joint goal of the region  $q$ ;
3 while  $\mathcal{R} \neq \emptyset$  do
   /* Sample a new root region */
4    $r \leftarrow \text{UniformSampling}(\mathcal{R})$ ;
5    $\mathcal{R}.\text{Remove}(r)$ ;
   /* Obtain a root goal in joint space */
6    $q_{\text{root}} \leftarrow \text{IK}(r, q_{\text{start}})$ ;
7   if  $q_{\text{root}} = \text{null}$  then
8     continue;
   /* Solve for a root path */
9    $\pi_{\text{root}} \leftarrow \text{Plan}(q_{\text{start}}, q_{\text{root}}, \mathcal{C}_{\text{free}}(r))$ ;
10  if  $\pi_{\text{root}} = \text{null}$  then
11    continue;
   /* Store the root motion and its index */
12   $i_{\text{root}} \leftarrow L.\text{Append}(\pi_{\text{root}})$ 
   /* Register the root region as covered by
      this root motion */
13   $\mathcal{M}(r) \leftarrow i_{\text{root}}, q_{\text{root}}$ ;
   /* Try to cover nearby regions via
      adaptation from the same root motion */
14  forall  $r_n \in \text{NearestNeighbor}(r, N_{\text{neighbor}})$  do
15     $q_n \leftarrow \text{IK}(r_n, q_{\text{root}})$ ;
16    if  $q_n = \text{null}$  then
17      continue;
18     $\pi_n \leftarrow \mathcal{A}.\text{Adapt}(\pi_{\text{root}}, q_n)$ ;
19    if  $\text{IsValidPath}(\pi_n) = \text{true}$  then
20       $\mathcal{M}(r_n) \leftarrow i_{\text{root}}, q_n$ ;
21       $\mathcal{R}.\text{Remove}(r_n)$ ;
22 return  $L, \mathcal{M}$ ;

```

After task space discretization, a brute-force library would simply plan and store a path for every TCR. This is often impractical due to its large memory footprint.

In line with library compression strategies [3], [7], we compress our library by planning root paths for a subset of TCRs and adapting them to cover additional regions, as summarized in Alg. 1. The algorithm takes TCRs $\mathcal{R}$, adaptation method $\mathcal{A}$ and starting configuration q_{start} as input. It initializes an empty root-motion list L and an empty hash

map $\mathcal{M}$ that associates each region r with a root motion Π_{root} and the joint goal of the region q (line 1-line 2).

While $\mathcal{R}$ is not empty and we have uncovered regions, the algorithm uniformly samples a region r from the TCRs $\mathcal{R}$ as root region and removes it from further consideration (line 3-line 5). It then solves inverse kinematics (IK), warm-started from q_{start} , to obtain a root goal configuration q_{root} for the task at r (line 6). If IK fails, the iteration terminates and continues to the next sample (line 7-line 8).

Given a feasible IK goal, a root joint-space path π_{root} is planned from q_{start} to q_{root} with free space $\mathcal{C}_{\text{free}}(r)$ conditioned on the TCR r (line 9), where collisions account for the swept volume induced by all possible goal poses within r [15]. In this work, we use RRT-Connect [9] to plan the initial solution and run path simplification and smoothing [18]. If planning fails, the task is considered infeasible and the algorithm continues to the next region (line 10-line 11). Otherwise, the resulting root path is added to the library L (line 12). The root region r is then covered by π_{root} , conditioned on q_{root} , and being registered in the map (line 13).

Finally, the algorithm attempts to expand region coverage by iterating over a set of N_{neighbor} nearest neighbors r_n around the root region (line 14). In this work, we choose $N_{\text{neighbor}} = 1000$. For each neighbor, it solves IK, warm-started from q_{root} , to obtain a joint goal q_n of the neighbor region (line 15-line 17). If successful, the adaptation module $\mathcal{A}$ generates a candidate path π_n by transforming the stored root motion π_{root} conditioned on q_n with method Adapt (line 18). If the adapted path can both reach the goal and is collision free, the neighbor region is marked as covered in $\mathcal{M}$ and removed from $\mathcal{R}$ (line 19-line 21). When $\mathcal{R}$ becomes empty, it returns the library L and the region-to-solution map $\mathcal{M}$ (line 22).

C. Adaptation Methods

All adaptations in COAD are designed to be fast and deterministic. To preserve efficiency, we propose to deliberately *exclude* obstacle terms from the adaptation objectives. Instead, collision checking is performed during offline verification (line 19 of Alg. 1), ensuring that the adapted path is collision-free. Below we evaluate three different adaptation strategies, each in terms of its general idea, what is stored as π_{root} , and how Adapt produces path π .

1) *Linear Interpolation (LI)*: For nearby goals, the simplest adaption strategy is to connect the end of the root path q_{root} with the desired goal q_{goal} through linear interpolation. Specifically, we construct a waypoint sequence $\pi_{\text{end}} = \{q_0, \dots, q_{T_{\text{end}}}\}$ of length T_{end} , whose first configuration is $q_0 = q_{\text{root}}$ and whose last configuration is $q_{T_{\text{end}}} = q_{\text{goal}}$:

$$q_n = (1 - \frac{n}{T_{\text{end}}})q_{\text{root}} + \frac{n}{T_{\text{end}}}q_{\text{goal}}, \quad n \in [1, T_{\text{end}}], \quad (9)$$

and append π_{end} to the root path π_{root} . This produces a joint-space path that reaches q_{goal} exactly. T_{end} can be chosen with the desired path resolution. Also, we include a continuity verification from [19] to avoid end-effector discontinuities.

2) *Dynamic Movement Primitives (DMPs)*: DMPs provide a compact dynamical system representation of a demonstrated

path and allow fast goal retargeting while preserving qualitative shape [20]. Given a path as reference, we model each joint path using a second-order stable dynamical system with a learned nonlinear forcing term:

$$\ddot{q}_i = \alpha_y (\beta_y (g_i - q_i) - \dot{q}_i) + f_i(s), \quad (10)$$

where $\alpha_y, \beta_y > 0$ are gain hyper-parameters controlling convergence and damping, g_i is the goal of each joint, i denotes the joint index, and $s \in [0, 1]$ is a phase variable for motion progression. The forcing term $f_i(s)$ is represented as a weighted sum of N_B fixed basis functions and is learned from the reference path.

For this method, instead of storing π_{root} we fit independent DMPs with π_{root} to each joint dimension and instead store the learned basis weights as the root motion representation π_{root} . In contrast to storing all T waypoints, DMPs require only $O(N_B)$ parameters per joint. The function $\mathcal{A}.\text{Adapt}$ performs a forward rollout of Eq. 10 with the goal set to the given q_{goal} , producing the adapted trajectory π .

3) *Simple Trajectory Optimization (STO)*: The STO adaptation performs a simplified trajectory refinement inspired by trajectory optimization via sequential convex optimization (TrajOpt) [10], and by the observation that stored motions can serve as effective warm-starts for optimization [12]. Given the stored root path π_{root} as initialization, we solve a single convex quadratic program that minimizes joint-space velocity and acceleration while penalizing deviation from the stored root path, subject to boundary and joint-limit constraints. Let $q_{0:T} := (q_0, \dots, q_T)$ denote the path variables, and let $q_{\text{root},0:T}$ denote the waypoints of the root path. The QP [21] solves:

$$\begin{aligned} \min_{q_{0:T}} \quad & w_v \|D_1 q_{0:T}\|_2^2 + w_a \|D_2 q_{0:T}\|_2^2 + w_s \|q_{0:T} - q_{\text{root},0:T}\|_2^2 \\ \text{s.t.} \quad & q_0 = q_{\text{start}}, \quad q_T = q_{\text{goal}}, \quad q_{\min} \leq q_t \leq q_{\max} \quad \forall t, \end{aligned} \quad (11)$$

where $w_v, w_a, w_s > 0$ are weighting parameters, D_1 and D_2 denote first- and second-order finite-difference operators along the path. The function $\mathcal{A}.\text{Adapt}$ solves the convex QP in Eq. 11, and returns an optimized path π satisfying the desired new goal q_{goal} .

D. Online Queries

Algorithm 2: Query($\mathbf{T}_o, \mathcal{M}, L, \mathcal{A}$)

```

/* Find TCR region */
1  $r \leftarrow \text{GetRegion}(\mathbf{T}_o)$ ;
/* Acquire joint goal and root motion */
2 if  $r \notin \mathcal{M}$  then
3   return null;
4  $i_{\text{root}}, q_{\text{goal}} \leftarrow \mathcal{M}(r)$ ;
5  $\pi_{\text{root}} \leftarrow L(i_{\text{root}})$ ;
/* Apply adaptation */
6  $\pi_{\text{sol}} \leftarrow \mathcal{A}.\text{Adapt}(\pi_{\text{root}}, q_{\text{goal}})$ ;
7 return  $\pi_{\text{sol}}$ ;

```

COAD answers an online query by retrieving and adapting a stored root motion from the compressed library, as summarized in Alg. 2. Given a sensed object pose $\mathbf{T}_o \in \mathcal{T}$, the

procedure first computes its associated TCR index (Eq. 8) to find TCR region r (line 1). If the index is not covered by the map, the query terminates and returns null, indicating that solution is not available for this problem (line 2 - line 3). Otherwise, the map returns the corresponding root motion π_{root} together with the goal joint configuration q_{goal} for this problem (line 5). Finally, the adaptation module $\mathcal{A}$ transforms π_{root} into a joint-space path conditioned on q_{goal} , producing the solution π_{sol} (line 6-line 7).

The online phase therefore reduces planning to a constant-time retrieval followed by lightweight goal-conditioned deformation. The online cost is dominated by two components: (i) TCR indexing and retrieval, and (ii) adaptation cost.

As shown in Eq. 8, the acquisition of TCR index and TCR region r is a fixed $O(1)$ cost arithmetic operation, and the retrieval of π_{root} from $\mathcal{M}$ and L are expected $O(1)$ with hash lookup and array indexing.

Among the adaptation methods, linear interpolation is $O(T_{\text{end}})$, where T_{end} is the number of interpolated waypoints from the root end to the desired goal. In this work, we use $T_{\text{end}} = 10$. The time complexity of DMP rollout is $O(T \cdot N_B)$ with fixed B basis functions and T generated waypoints. For the optimization-based adapter STO, the runtime scales approximately as $O(T \cdot I)$, where I denotes the max number of iterations. In motion planning practice, where trajectories are stored at a fixed resolution (in this work, $T = 200$), this yields a predictable fixed runtime. Since all adapter parameters, such as T_{end} , T , N_B and I , are fixed offline, the online query time is constant with respect to the online input.

V. EXPERIMENTS

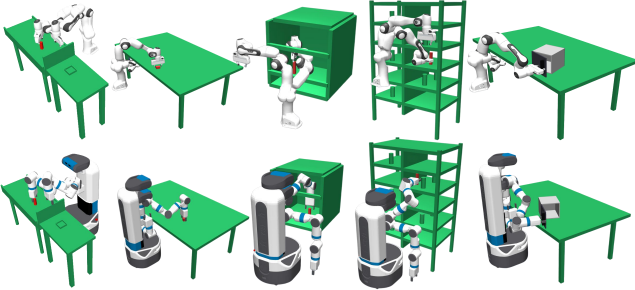


Fig. 4: Panda (top) and Fetch (bottom) manipulators planning in Conveyor, All Stable, Cage, Shelf, Microwave environments (from left to right) [22].

A. Experimental Setup

We evaluate the performance of COAD on two manipulators: Panda (7-DOF) and Fetch (8-DOF) in five different environments adapted from [22] and shown in Fig. 4

- **Conveyor:** A fixed conveyor belt with a rectangular cuboid object that can arrive randomly. Goal variation parameters: $x_i \pm 0.2$ m, $y_i \pm 0.4$ m, $\theta_i \pm \frac{\pi}{2}$ rad.
- **All Stable:** A fixed table with a single asymmetric rectangular cuboid that can be placed stably on any of its six faces. Goal variation parameters: $x_i \pm 0.6$ m, $y_i \pm 1.0$ m, $\theta_i \pm \frac{\pi}{2}$ rad, where $i \in \{1, \dots, 6\}$ denotes the placement face.

TABLE I: Plan Library (OOM denotes Out of Memory)

Robot -Env.	Metric	Full Library	CoAd -LI	CoAd -DMP	CoAd -STO
Panda -Conveyor	Comp. (%)	-	98.72	99.41	98.73
	Time (ms)	0.042	0.053	0.894	3.175
	Quality (rad)	1.47	2.18	1.53	2.14
	Size	13974	179	82	178
Panda -All Stable	Comp. (%)	-	74.39	34.10	93.82
	Time (ms)	0.039	0.058	1.150	5.493
	Quality (rad)	5.31	5.48	5.18	5.42
	Size	74107	18982	48840	4580
Panda -Cage	Comp. (%)	-	78.37	63.41	80.83
	Time (ms)	0.035	0.054	1.215	4.322
	Quality (rad)	7.93	7.95	7.26	8.03
	Size	6693	1448	2449	1283
Panda -Shelf	Comp. (%)	-	79.03	56.69	82.19
	Time (ms)	0.039	0.058	1.240	4.483
	Quality (rad)	4.78	5.03	4.83	4.95
	Size	23290	4885	10088	4148
Panda -Microwave	Comp. (%)	-	91.37	OOM	94.99
	Time (ms)	OOM	0.084	OOM	6.186
	Quality (rad)	OOM	5.95	OOM	5.94
	Size	6414010	553801	OOM	320894
Fetch -Conveyor	Comp. (%)	-	98.35	98.33	98.41
	Time (ms)	0.043	0.054	0.949	3.538
	Quality (rad)	3.78	4.39	2.77	4.00
	Size	10177	168	170	162
Fetch -All Stable	Comp. (%)	-	62.77	9.58	71.29
	Time (ms)	0.042	0.062	1.556	5.387
	Quality (rad)	11.78	11.83	11.58	11.75
	Size	38480	14327	34793	11046
Fetch -Cage	Comp. (%)	-	65.89	38.21	72.91
	Time (ms)	0.043	0.066	3.761	10.134
	Quality (rad)	18.0	17.11	16.27	17.30
	Size	683	233	422	185
Fetch -Shelf	Comp. (%)	-	81.74	53.60	83.67
	Time (ms)	0.041	0.061	1.333	4.395
	Quality (rad)	7.68	7.69	7.14	7.56
	Size	36373	6641	16876	5938
Fetch -Microwave	Comp. (%)	-	78.08	OOM	80.62
	Time (ms)	OOM	0.082	OOM	9.535
	Quality (rad)	OOM	16.29	OOM	16.25
	Size	929441	203716	OOM	180082
UR10 -Real	Comp. (%)	-	93.54	69.99	95.22
	Time (ms)	0.042	0.061	1.502	4.615
	Quality (rad)	6.43	6.55	6.61	6.64
	Size	4415	285	1325	211

- **Cage:** The *cage* environment [22], with one rectangular goal object. Goal variation parameters: $x \pm 0.35$ m, $y \pm 0.35$ m, $\psi \pm \frac{\pi}{2}$ rad.
- **Shelf:** The *thin-shelf* environment [22], with five fixed obstacles and one rectangular goal object. Goal variation parameters: $x \pm 0.31$ m, $y \pm 0.5$ m, $\theta \pm \frac{\pi}{2}$ rad, $z \in \{0.35, 0.60, 0.85, 1.10, 1.35\}$ m.
- **Microwave:** A fixed table with an articulated microwave whose door configuration also varies across queries. Goal variation parameters: $x \pm 0.8$ m, $y \pm 0.8$ m, $\psi \pm \frac{\pi}{2}$ rad, $\theta_d \in [0, \frac{\pi}{2}]$ rad, where θ_d is the microwave door angle.

We perform a real world experiment with a physical UR10 robot shown in Fig. 1 and described in Sec. V-D.

In this experimental setting, the robots are tasked with reaching a desired end-effector pose corresponding to a grasp pose for the object. Each query specifies a goal pose sampled from the potential task space. In our setting, the static environment obstacles (e.g., conveyor, shelf, cage structure) remain fixed, while the query-dependent object to be grasped

TABLE II: Path Quality and Success Rate

Robot - Env.	Metric	RRT-Connect	RRT-Connect -VAMP	ERT-Connect	CoAD -LI	CoAD -DMP	CoAD -STO
Panda - Conveyor	Success (%)	100.00	100.00	100.00	100.00	100.00	100.00
	Quality (rad)	5.36 $\pm$ 0.80	1.90 $\pm$ 1.18	2.65 $\pm$ 1.88	2.18 $\pm$ 0.72	1.53 $\pm$ 0.57	2.15 $\pm$ 0.70
Panda - All Stable	Success (%)	100.00	100.00	100.00	100.00	100.00	100.00
	Quality (rad)	8.97 $\pm$ 2.84	8.53 $\pm$ 1.33	5.77 $\pm$ 1.47	5.42 $\pm$ 1.14	5.17 $\pm$ 1.04	5.42 $\pm$ 1.13
Panda - Cage	Success (%)	71.40	99.90	82.80	100.00	100.00	100.00
	Quality (rad)	20.50 $\pm$ 6.55	17.65 $\pm$ 6.50	9.39 $\pm$ 2.80	8.00 $\pm$ 2.26	7.27 $\pm$ 2.04	7.91 $\pm$ 2.26
Panda - Shelf	Success (%)	94.50	100.00	98.80	100.00	100.00	100.00
	Quality (rad)	11.91 $\pm$ 5.57	9.13 $\pm$ 3.87	6.40 $\pm$ 2.51	4.98 $\pm$ 1.47	4.82 $\pm$ 1.40	4.93 $\pm$ 1.38
Panda - Microwave	Success (%)	100.00	100.00	100.00	100.00	OOM	100.00
	Quality (rad)	10.27 $\pm$ 4.33	8.63 $\pm$ 2.20	6.89 $\pm$ 2.53	5.90 $\pm$ 1.69	OOM	5.93 $\pm$ 1.74
Fetch - Conveyor	Success (%)	100.00	98.10	99.90	100.00	100.00	100.00
	Quality (rad)	11.10 $\pm$ 3.56	4.15 $\pm$ 4.16	5.43 $\pm$ 4.67	4.55 $\pm$ 4.02	2.70 $\pm$ 2.49	3.94 $\pm$ 3.45
Fetch - All Stable	Success (%)	63.60	100.0	99.80	100.00	100.00	100.00
	Quality (rad)	34.99 $\pm$ 9.45	21.60 $\pm$ 3.75	13.73 $\pm$ 3.00	11.88 $\pm$ 2.63	11.63 $\pm$ 2.55	11.75 $\pm$ 2.57
Fetch - Cage	Success (%)	32.10	83.50	67.90	100.00	100.00	100.00
	Quality (rad)	39.51 $\pm$ 9.70	31.09 $\pm$ 7.33	18.05 $\pm$ 4.72	17.14 $\pm$ 4.54	16.41 $\pm$ 4.76	17.34 $\pm$ 4.22
Fetch - Shelf	Success (%)	68.40	99.80	95.10	100.00	100.00	100.00
	Quality (rad)	21.23 $\pm$ 9.67	15.11 $\pm$ 6.83	9.22 $\pm$ 3.60	7.58 $\pm$ 3.05	7.05 $\pm$ 2.89	7.54 $\pm$ 3.04
Fetch - Microwave	Success (%)	88.90	95.00	95.10	100.00	OOM	100.00
	Quality (rad)	39.83 $\pm$ 10.19	21.46 $\pm$ 4.78	18.20 $\pm$ 4.34	16.42 $\pm$ 4.21	OOM	16.15 $\pm$ 4.48
UR10 - Real	Success (%)	98.00	88.00	99.00	100.00	100.00	100.00
	Quality (rad)	19.69 $\pm$ 9.26	8.42 $\pm$ 4.68	7.15 $\pm$ 5.07	6.48 $\pm$ 3.96	6.60 $\pm$ 3.40	6.80 $\pm$ 4.28

varies across tasks. All experiments are conducted in Mujoco simulation on a machine with Intel i7-14700 and 32 GB RAM.

For each robot–environment pair, we first generate all the TCRs, followed by inverse kinematics (IK) to obtain joint-space goal configurations. The full solution library is generated by solving and storing each TCR. This full library serves as the uncompressed baseline. Compressed plan libraries are then generated with the three adaptation methods described in Sec. IV-C. We denote them as CoAD-LI, CoAD-DMP and CoAD-STO. We measure path quality using joint-space path length.

B. Offline Library Building

We first evaluate the compression performance of CoAD with three different adaptation methods. Table I summarizes the compression ratio, defined as the percentage reduction in the number of stored paths relative to the full library, together with the average adaptation time and the resulting path quality.

For both Panda-Microwave and Fetch-Microwave, the Full Library and CoAD-DMP exceeded the available memory on our 32 GB machine. Their measurements are therefore marked OOM (Out of Memory).

Among the adaptation methods, CoAD-STO generally achieves the highest compression, reducing the library size by 71–99%. CoAD-LI achieves similar compression of 63–99%. CoAD-DMP generally has a lower compression rate, achieving 10–99% compression across other problems. Where evaluated, CoAD-DMP produces shorter paths than CoAD-LI in simulation, whereas CoAD-LI produces the shortest paths among the compressed methods in the UR10 experiment.

C. Online Planning

We evaluate the online planning performance over 1000 random queries per setting. The quantitative results for success rate and path quality are summarized in Table II. CoAD is benchmarked against three baselines: RRT-Connect [9],

RRT-Connect-VAMP [23], and ERT-Connect [6]. VAMP accelerates sampling-based planning through vectorized forward kinematics and collision checking. We use its RRT-Connect implementation as a high-performance online planning baseline. Also, we compare against ERT-Connect [6], a representative experience-based method [4]–[6], [12] that follows a similar retrieve-and-repair mechanism. Libraries are constructed with N object poses sampled from the task set, where N is the largest compressed library size produced by CoAD for each problem, capped at 50,000 due to memory limitations. All methods are executed with a timeout of 3 seconds.

Across all robot–environment pairs, CoAD achieves a 100% success rate, while RRT-Connect, VAMP-RRTConnect and ERT-Connect occasionally fail in more constrained environments. Some VAMP-RRTConnect failures arise from its sphere-based over-approximation of the robot geometry, that make some IK solutions invalid. In terms of path quality, all CoAD adaptations outperform RRT-Connect and RRT-Connect-VAMP while achieving comparable quality to ERT-Connect, with CoAD-DMP providing the best path quality across most experiments.

Fig. 5 shows the distribution of planning times. Among the adaptation methods, CoAD-LI is the fastest, consistently achieving planning times in the range of 50-90 microseconds. CoAD-DMP and CoAD-STO achieve planning times of 1-10 milliseconds. In terms of planning speed, CoAD-LI significantly outperforms RRT-Connect and ERT-Connect, achieving up to two orders of magnitude speedup in the open tabletop environments and nearly three orders of magnitude in more constrained environments such as Cage and Shelf. RRT-Connect-VAMP achieves faster planning times in some easier Panda experiments like Panda-Conveyor and Panda-All Stable but struggles to outperform CoAD when planning complexity increases in the higher DOF Fetch experiments.

In addition to its low mean planning time, CoAD exhibits

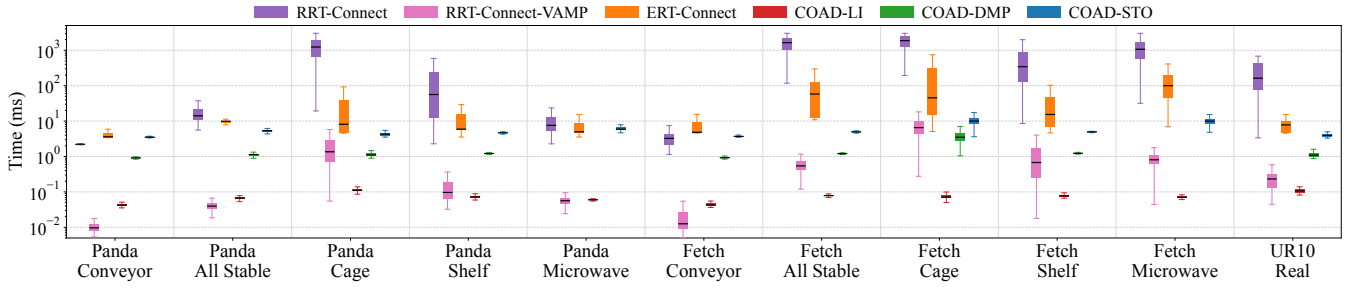


Fig. 5: Comparison of Planning Times for all methods. Y-axis is in log-scale milliseconds.

substantially smaller planning-time variance than the other baselines across environments. This predictable runtime empirically supports the constant-time online complexity derived in Sec. IV-D.

D. Real-world Validation

To validate practical applicability, we evaluate with a UR10 robot in a table scenario as shown in Fig. 1. The compressed library is generated in simulation and transferred directly to the physical system. We assume that the geometry and poses of the fixed obstacles, as well as those of the goal object, are known a priori. During execution, the pose of the goal object is estimated online using an AprilTag-based vision system.

We randomly placed the goal object within the predefined task space for 100 trials. The observed performance trends, including success rates, query time, and path quality, are consistent with the simulation results and are reported in Fig. 5 and Table II.

VI. CONCLUSION AND FUTURE WORK

In this paper, we present COAD, a framework for goal-varying manipulation planning that combines task-space discretizations via TCRs, offline construction of a compressed library, and efficient online adaptation. By shifting the majority of the planning and adaptation verification effort to an offline phase, COAD enables constant-time query while significantly reducing memory requirements.

Currently, the root regions are randomly selected. A promising direction for future work is to design more representative root-selection algorithms to further minimize the number of root paths required. Another important extension is to relax the semi-static assumption and build a motion library suitable for more general semi-static environments, which includes both varying goals and moving obstacles [15].

REFERENCES

- [1] H. Choset, K. M. Lynch, S. Hutchinson, G. A. Kantor, and W. Burgard, *Principles of robot motion: theory, algorithms, and implementations*. MIT press, 2005.
- [2] K. E. Bekris, J. Doerr, P. Meng, and S. Tangirala, “The state of robot motion generation,” *arXiv preprint arXiv:2410.12172*, 2024.
- [3] H. Ishida, N. Hiraoka, K. Okada, and M. Inaba, “Coverlib: Classifiers-equipped experience library by iterative problem distribution coverage maximization for domain-tuned motion planning,” *IEEE Trans. Robot.*, 2025.
- [4] D. Berenson, P. Abbeel, and K. Goldberg, “A robot path planning framework that learns from experience,” in *IEEE Intl. Conf. Robot. Autom.*, 2012, pp. 3671–3678.
- [5] D. Coleman, I. A. Şucan, M. Moll, K. Okada, and N. Correll, “Experience-based planning with sparse roadmap spanners,” in *IEEE Intl. Conf. Robot. Autom.*, 2015, pp. 900–905.
- [6] E. Pairet, C. Chamzas, Y. Petillot, and L. E. Kavraki, “Path planning for manipulation using experience-driven random trees,” vol. 6, no. 2, pp. 3295–3302.
- [7] F. Islam, O. Salzman, A. Agarwal, and M. Likhachev, “Provably constant-time planning and replanning for real-time grasping objects off a conveyor belt,” *Intl. J. of Robotics Research*, vol. 40, no. 12-14, pp. 1370–1384, 2021.
- [8] L. E. Kavraki, M. N. Kolountzakis, and J.-C. Latombe, “Analysis of probabilistic roadmaps for path planning,” *IEEE Trans. Robot.*, vol. 14, no. 1, pp. 166–171, 1998.
- [9] J. Kuffner and S. LaValle, “Rrt-connect: An efficient approach to single-query path planning,” in *IEEE Intl. Conf. Robot. Autom.*, vol. 2, 2000, pp. 995–1001 vol.2.
- [10] J. Schulman, J. Ho, A. Lee, I. Awwal, H. Bradlow, and P. Abbeel, “Finding locally optimal, collision-free trajectories with sequential convex optimization,” in *Robotics: Science and Syst.*, 2013.
- [11] N. Ratliff, M. Zucker, J. A. Bagnell, and S. Srinivasa, “Chomp: Gradient optimization techniques for efficient motion planning,” in *IEEE Intl. Conf. Robot. Autom.*, 2009, pp. 489–494.
- [12] T. S. Lembono, A. Paolillo, E. Pignat, and S. Calinon, “Memory of motion for warm-starting trajectory optimization,” *IEEE Robot. Autom. Letters*, vol. 5, no. 2, pp. 2594–2601, 2020.
- [13] C. Chamzas, Z. Kingston, C. Quintero-Peña, A. Shrivastava, and L. E. Kavraki, “Learning Sampling Distributions Using Local 3D Workspace Decompositions for Motion Planning in High Dimensions,” in *IEEE Intl. Conf. Robot. Autom.*, June 2021, pp. 1283–1289.
- [14] F. Islam, C. Paxton, C. Eppner, B. Peele, M. Likhachev, and D. Fox, “Alternative paths planner (app) for provably fixed-time manipulation planning in semi-structured environments,” in *IEEE Intl. Conf. Robot. Autom.*, 2021, pp. 6534–6540.
- [15] N. K. Ilampooranan and C. Chamzas, “Cover: Coverage-verified roadmaps for fixed-time motion planning in continuous semi-static environments,” *arXiv preprint arXiv:2510.03875*, 2025.
- [16] J. P. Van Den Berg and M. H. Overmars, “Roadmap-based motion planning in dynamic environments,” *IEEE Trans. Robot.*, vol. 21, no. 5, pp. 885–897, 2005.
- [17] D. Berenson, S. Srinivasa, and J. Kuffner, “Task space regions: A framework for pose-constrained manipulation planning,” *Intl. J. of Robotics Research*, vol. 30, no. 12, pp. 1435–1460, 2011.
- [18] I. A. Şucan, M. Moll, and L. E. Kavraki, “The Open Motion Planning Library,” *IEEE Robot. Autom. Magazine*, vol. 19, no. 4, pp. 72–82, December 2012, .
- [19] Z. Zhong, Z. Li, and C. Chamzas, “Expansion-GRR: Efficient generation of smooth global redundancy resolution roadmaps,” in *IEEE/RSJ Intl. Conf. on Intell. Robots and Syst.*, 2024, pp. 8854–8860.
- [20] M. Saveriano, F. J. Abu-Dakka, A. Kramberger, and L. Peternel, “Dynamic movement primitives in robotics: A tutorial survey,” *Intl. J. of Robotics Research*, vol. 42, no. 13, pp. 1133–1184, 2023.
- [21] B. Stellato, G. Banjac, P. Goulart, A. Bemporad, and S. Boyd, “OSQP: an operator splitting solver for quadratic programs,” *Mathematical Programming Computation*, vol. 12, no. 4, pp. 637–672, 2020.
- [22] C. Chamzas, C. Quintero-Peña, Z. Kingston, A. Orthey, D. Rakita, M. Gleicher, M. Toussaint, and L. E. Kavraki, “Motionbenchmarker: A tool to generate and benchmark motion planning datasets,” *IEEE Robot. Autom. Letters*, vol. 7, no. 2, p. 882–889, Apr. 2022.
- [23] W. Thomason, Z. Kingston, and L. E. Kavraki, “Motions in microseconds via vectorized sampling-based planning,” in *IEEE Intl. Conf. Robot. Autom.* IEEE, 2024, pp. 8749–8756.